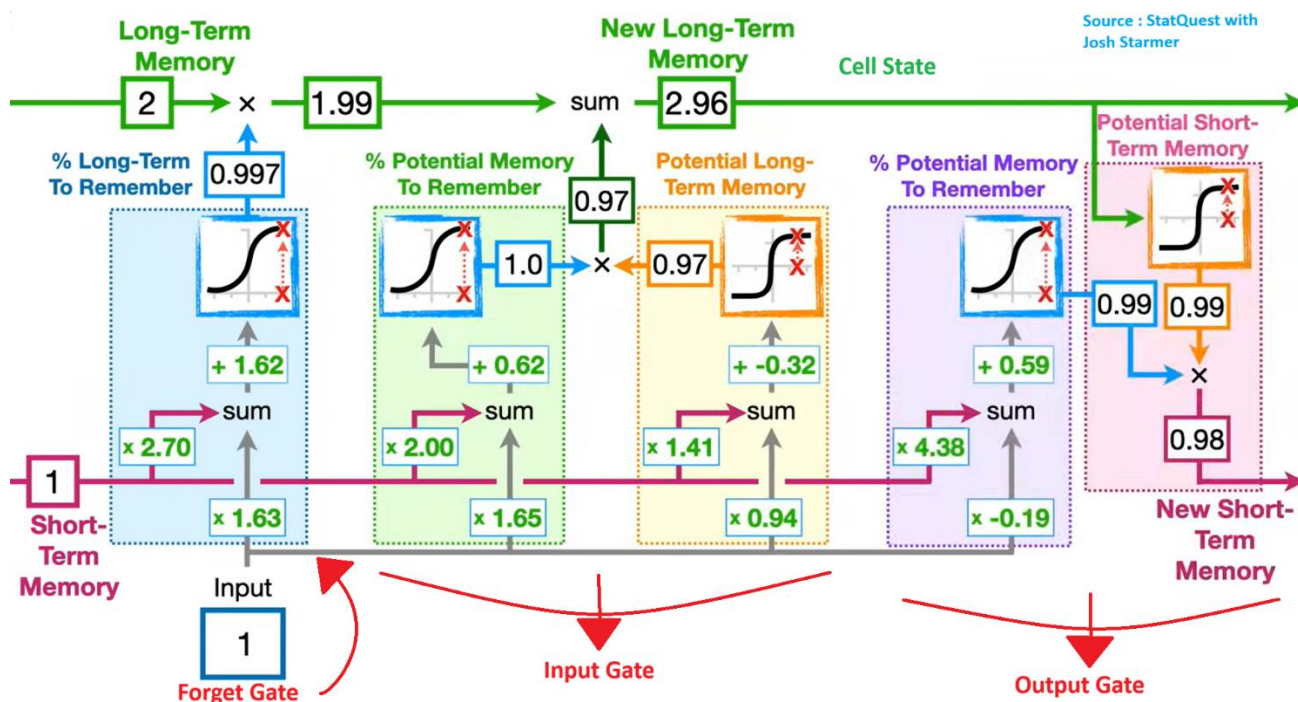


1. The Role of Each Gate :

The core advantage of this architecture is its ability to selectively remember and discard meteorological patterns over a 72-hour window using three distinct gates:

- **The Forget Gate:** This gate processes the current weather input and the previous hidden state through a sigmoid function. It essentially outputs a number between **0** and **1** for each piece of data in the cell state. A **0** forces the model to completely "forget" irrelevant history (like a temperature spike from two days ago that no longer matters), while a **1** retains crucial overarching trends.
- **The Input Gate & Candidate State:** The network calculates new potential information (for the candidate state using a tanh layer) and uses the Input gate's sigmoid layer to decide exactly *how much* of this new data should be added to the memory.
- **The Output Gate:** Finally, the model decides what information needs to be passed to the next time step. It filters the updated cell state through a tanh function and multiplies it by the Output gate's sigmoid layer, yielding the hidden state (**h_t**) which carries the predictive signal forward.



THE 5 GOVERNING EQUATIONS OF LSTM

Forget Gate (f_t): $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

Input Gate (i_t): $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

Candidate State ($\tilde{C}_t$): $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$

Cell State Update (C_t): $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output Gate & Hidden State (h_t): $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \rightarrow h_t = o_t * \tanh(C_t)$

2. Retaining and Forgetting Information (The Cell State)

The fundamental reason this architecture succeeds where standard RNNs fail is the **Cell State (C_t)**. It acts as a continuous belt running straight down the entire 72-hour sequence. Because the forget and input gates interact with the cell state using simple linear addition and multiplication, the gradients can flow freely during backpropagation without vanishing, allowing the network to retain a stable memory of the 3 days (72 hours) temperature cycle.

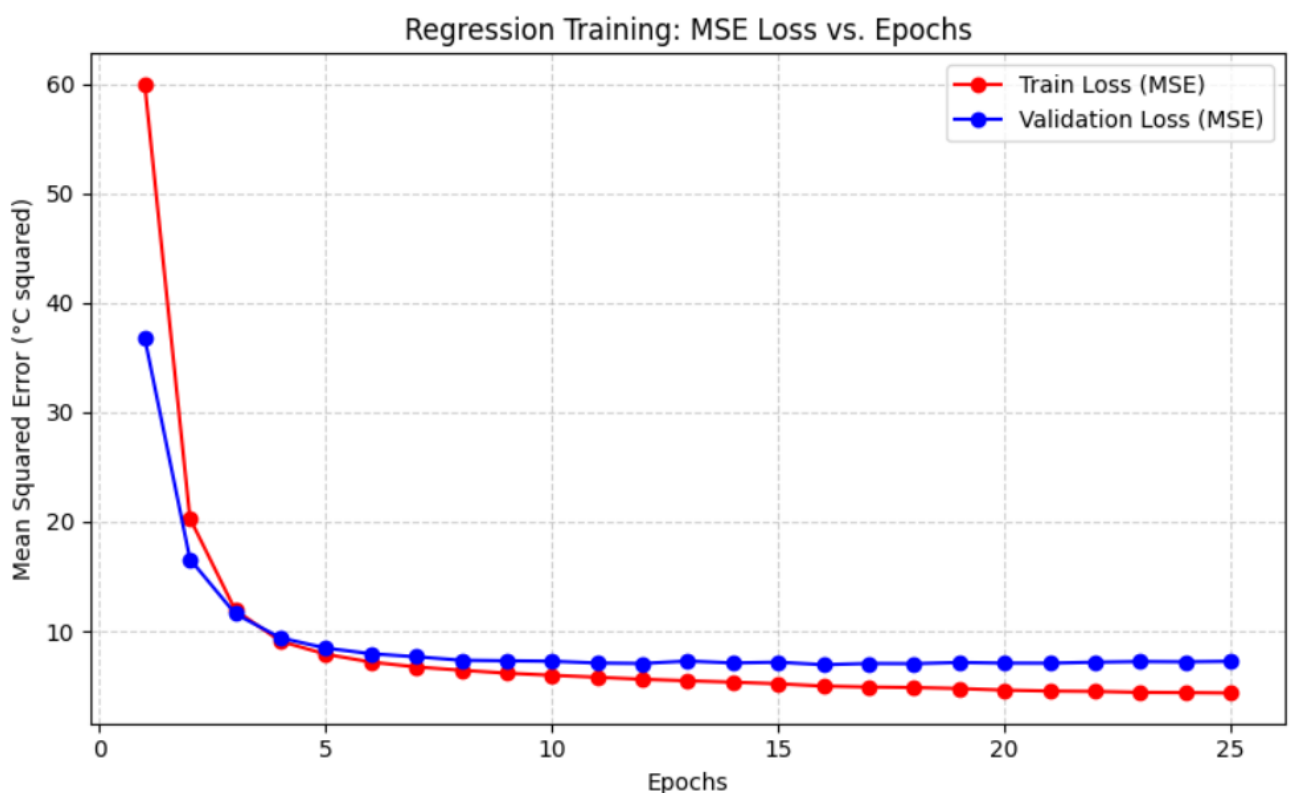
3. Training Stability

The model demonstrated excellent training stability on a regression objective. Utilizing the Adam optimizer and an MSE loss function, the model achieved perfect convergence over 25 epochs. The Training Loss and Validation Loss curves tracked tightly together, indicating that the model generalized effectively without overfitting the historical weather patterns.

- **Stochastic Batch Shuffling:** By shuffling the 72-hour sliding windows during training, the model was prevented from memorizing the specific chronological sequence of the Jena dataset.
- To further refine the model's convergence, a **ReduceLROnPlateau learning rate scheduler** was integrated into the training loop. By actively monitoring the Validation Loss, the scheduler dynamically

reduced the learning rate by half whenever the loss plateaued for two consecutive epochs.

- **Optimal Weight Initialization:** PyTorch's random weight initialization placed the final model in a favorable optimization landscape. The final training run converged smoothly, achieving a stable :
 - Squared Error (MSE): 7.2505
 - Mean Absolute Error (MAE): 2.1016 °C
 - Huber Loss: 1.6521.



4. Sequence Length Considerations

The raw Jena Climate dataset is recorded at **10** minute intervals (**6** observations per hour). To forecast based on a **72** hour window, utilizing the raw data would require a sequence length of **T = 432** time steps. This approach was avoided and subsampling the data to 1-hour intervals (**T = 72**) was done due to :

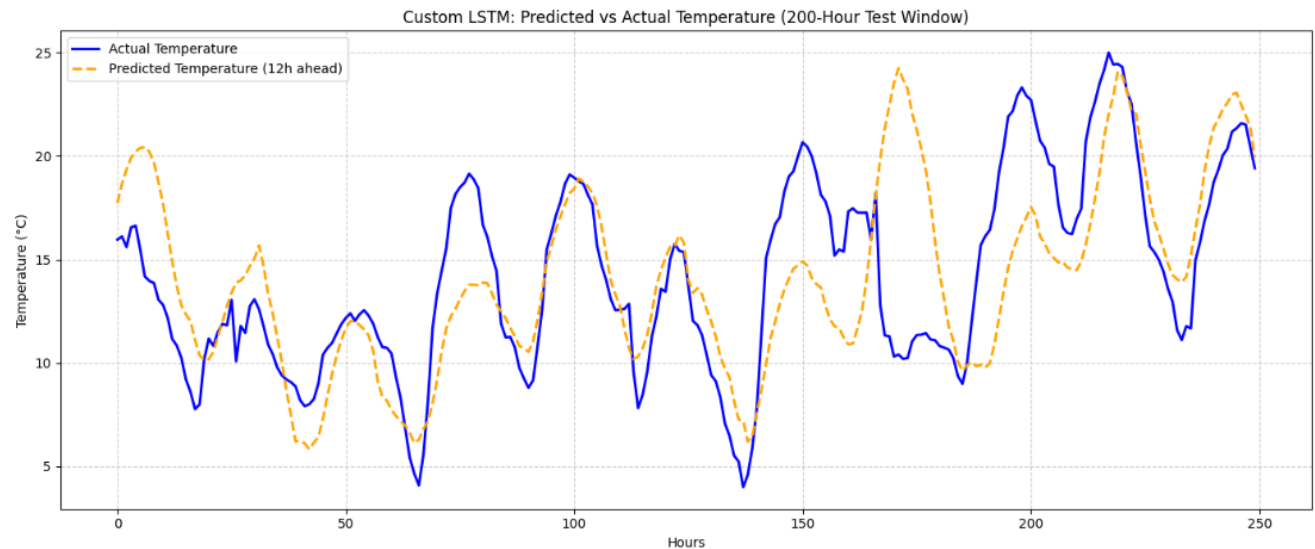
- **Signal Isolation:** Atmospheric temperature changes are macroscopic. 10-minute intervals introduce unnecessary meteorological noise (e.g. momentary wind gusts). Subsampling to 1 hour intervals acts as a low pass filter, isolating the actual diurnal temperature cycle.
- **Gradient Preservation:** While LSTMs are designed to mitigate vanishing gradients, backpropagating through 432 unrolled recurrent steps in a custom, architecture risks severe gradient decay. A 72 step sequence ensures robust gradient flow through the forget and input gates across the entire temporal window.
- **Computational Efficiency:** Reducing the sequence length by a factor of 6 drastically reduced the memory utilized during the forward pass and accelerated training convergence without reducing predictive context.

5. Forecasting Challenges and Model Limitations

Deep learning based timeseries forecasting faces distinct hurdles when applied to real world chaotic systems like atmospheric weather:

- **Lag and Reversion to Mean Bias:** LSTMs lean heavily on recent historical trends to predict immediate future states. When a temperature sequence reaches a historical peak or plateau (as seen in the incorrect validation samples), the network naturally predicts a downward reversion to the mean (Colder/Same). However, sudden atmospheric changes can cause localized, non linear temperature spikes that entirely violate historical patterns, causing the model to commit errors
- **Accumulation of Errors over Long Sequences:** LSTMs struggle with long-range dependencies when hidden states are stretched across massive timelines. While our 72 hour historical window successfully mitigates this, attempting to extend the target horizon (e.g. predicting 24 or 48 hours out with this architecture) would cause severe degradation in predictive reliability due to unpredictable atmospheric drift.

--- Final Regression Metrics ---
Mean Squared Error (MSE): 7.2505
Mean Absolute Error (MAE): 2.1016 °C
Huber Loss: 1.6521



6. Explanation of Design Choices :

- Direct Horizon Continuous Regression :** To satisfy the requirement of predicting the exact temperature 12 hours into the future, the model was structured as a continuous regressor. The output layer was reduced to a single neuron (outputs=1), and the target labels were set to the raw, unscaled Celsius values of the 84th hour. To train this continuous output, Mean Squared Error (MSE) was utilized as the loss function, natively penalizing the model proportional to the magnitude of its specific temperature error.
- Manual Gate Implementation (not using nn.LSTM):** Because built in modules were forbidden, the architecture utilized individual linear transformations (W_f , W_i , W_c , W_o) for the input, forget, and output gates. The primary challenge here was unrolling the temporal loop in the forward() pass and managing the hidden state (h_t) and cell state (C_t) across 72 discrete steps, ensuring PyTorch's autograd could still track the computational graph for backpropagation.
- Hidden Dimension (Size = 64):** The constraints required a hidden dimension between 32 and 256. A size of **64** was selected. A minimum dimension of 32 risks an information bottleneck, preventing the network

from effectively compressing 14 atmospheric features across 72 hours into a single memory vector. Conversely, pushing the dimension to 256 quadratically inflates the size of the weight matrices. Given the manual for loop implementation, a hidden size of 256 would have drastically slowed down training on the GPUs and exponentially increased the risk of the model memorizing the training data (overfitting). **64** provided the optimal balance of memory capacity and computational efficiency.

- **Layer Depth (Single Layer):** While the constraints permitted up to 2 stacked LSTM layers, the model was constrained to a single layer. In computer vision, stacking layers extracts deep hierarchical features (like pixels to edges to objects). However, in time-series meteorological forecasting, temporal data rarely benefits from deep vertical abstraction. A second stacked LSTM layer would have doubled the computational requirements and severely complicated the manual gradient flow without providing significant predictive improvements.